NatWest Group

present Code for Purpose – India Hackathon

Submission guidelines & terms

These guidelines set out the expectations, standards and rules for student project submissions. Please read them carefully and ensure your project complies with all sections before submission.

1. Required submission items

1.1 README.md (Project Documentation)

Your README.md is the main documentation for your project. It must be clear enough for someone unfamiliar with your work to understand and run it.

Your README must include:

i. Overview

A short description (2–5 sentences) explaining:

- What your project does.
- What problem it solves.
- Who the intended users are.

ii. Features

- A bullet list of features that are implemented and working.
- Do not list planned or future features as if they already exist.

iii. Install and run instructions

- Step-by-step instructions to install dependencies and run the project.
- Assume the reader has basic technical skills but no prior knowledge of your code.

iv. Tech stack

List the main technologies used, e.g.,:

- Programming languages.
- Frameworks.
- Databases.
- Cloud services.
- AI/ML libraries or models.

NatWest Group

v. **Usage examples**

- Show how to use the project once it is running.
- You may include:
- Command examples
- Example API calls
- Sample inputs/outputs
- Screenshots (if applicable)

vi. **Optional but recommended sections**

- Architecture notes: short explanation of how the system is structured (e.g., frontend, backend, database, external APIs).
- Limitations: honest description of what does not work or is not fully implemented.
- Future improvements: features or changes you would make with more time.

---

2. Codebase requirements

Your repository must contain everything needed to understand and run your project.

2.1 Complete source code

- Include all source files used by your project (e.g., .py, .js, .ts, .html, .css, etc.).
- Ensure no imported modules or files are missing.
- Test by cloning your repository to a new location and following your own setup instructions.

2.2 Dependency files

Provide the necessary configuration for installing dependencies, for example:

- Python: requirements.txt or pyproject.toml
- Node.js: package.json (and optionally package-lock.json or pnpm-lock.yaml/yarn.lock)
- Conda: environment.yml

These files must allow judges to install dependencies with standard commands such as:

- pip install -r requirements.txt
- npm install
- conda env create -f environment.yml

2.3 Configuration files

Include configuration examples without exposing secrets.

- .env.example should list all required environment variables.
- Do not include real passwords, API keys or confidential data.
  You may also include:
  - Configuration folders (e.g., config/).
  - Settings files (e.g., settings.py, config.json).

2.4 Folder structure

Avoid single-file projects. Use a clear and logical structure, for example:

project/

- src/
- tests/
- assets/
- README.md
- requirements.txt

Additional common directories:

- docs/ for extra documentation.
- scripts/ for helper or utility scripts.

A clean structure is required to help judges navigate your project quickly.

3. Tests (optional but encouraged)

Tests are optional but required if you want to be assessed on test coverage.

Place tests in a tests/directory, or:

- Use filenames like test_*.py (Python).
- Or *.test.js / *.spec.js (JavaScript/TypeScript).

Tests must be real and meaningful, not empty placeholders.

---

4. Feature accuracy and honesty

Your documentation must reflect the actual state of your project.

- Every feature listed in the README must exist and be usable in the codebase.
- Partially implemented or incomplete features must be clearly labelled as such.

Examples of accurate descriptions:

- "User registration is implemented with a basic form; email verification is not yet supported."
- "The dashboard page is present, but charts are static and not connected to live data."

Misrepresenting features or claiming work that has not been done may lead to disqualification.

---

5. Code quality standards

You are expected to follow basic open-source style and good engineering practices.

5.1 Structure and organisation

- Use a logical folder structure (see Section 2.4).
- Avoid placing all logic in a single file (e.g., main.py, index.js).

5.2 Naming and comments

- Use descriptive names for variables, functions and files (e.g., calculate_score, user_session, task_repository).
- Avoid meaningless names (e.g., temp1, x2, data1).

Include helpful comments and docstrings:

- Explain "why" something is done or any non-obvious logic.
- Add docstrings for important functions, classes and modules.

5.3 Security and secrets

- Do not hard code:
  - Usernames
  - Passwords
  - API keys
  - Tokens
- Use environment variables and reference them in code.
- Provide env.example instead of your real .env file.

5.4 Clean-up before submission

Before submitting:

- Remove unused files, temporary scripts and test data that are not needed.
- Remove debugging code (e.g., stray print statements, console logs).
- Remove local log files and output artefacts (e.g., debug.log, output.txt).

Readable, straightforward code is preferred over unnecessarily complex or "clever" solutions.

6. Highlighting technical depth (if applicable)

If your project uses advanced technologies
(e.g., AI, data pipelines, etc), you should:

- Provide a short explanation in the README:
  - What technologies or techniques you used.
  - Why you chose them.
  - What problem they solve in your project.
- Optionally include simple diagrams (inline in the README or in a docs/ folder), such as:
  - System architecture diagrams (e.g., client → backend → database → external API).
  - Data flow diagrams (e.g., input → processing → output).

Example explanation:

We use a fine-tuned transformer model to classify user support tickets by topic. This improves routing accuracy compared to keyword-based matching, which performed poorly in our tests.

This helps reviewers understand the complexity and innovation in your submission.

---

7. Open-source compliance and DCO terms

These terms are mandatory for participation. By submitting a project, you confirm that you comply with the following.

7.1 Github accounts and commit sign-off

You may use:

- Your personal GitHub account.

All commits must be compatible with:

- Apache License 2.0.
- Developer Certificate of Origin (DCO).

You are responsible for following the hackathon instructions on commit sign-off (for example, using git commit -s and including the correct sign-off text).

7.2 Single email rule

You must use a single email address for:

- All Git commits.
- All hackathon-related communication.
- The entire duration of the event.

This ensures consistent identification of your contributions.

7.3 Identity and representation

- Do not use fake identities, pseudonymous company personas or shared corporate GitHub users.
- All contributions are made in a personal capacity and not as official company work, unless explicitly authorised and disclosed.

7.4 Licensing, policies and conflicts of interest

You must:

- Comply with the open-source policies.
- Follow external open-source standards, including the DCO requirements.
- Not disclose or use confidential or proprietary company information in your project.
- Use personal devices for hackathon work unless company policy explicitly allows otherwise.

7.5 Repository visibility

- During the hackathon, your GitHub repository must remain private.
- After submission and review, repositories may be made public, as required by the event rules.

7.6 No plagiarism policy

- All project work must be original to you and your team.
- You must not copy other teams' work or reuse previous projects as if they were new submissions.
- Using open-source libraries and frameworks is allowed, but your project must add original code, configuration and integration work.
- You must respect licences of all third-party dependencies.

Violation of these terms may result in disqualification.

---

8. Recommended learning resources (optional)

The following external courses are recommended to improve your understanding of open source, but they are not mandatory.

i. Open-source contribution in finance (LFD137)

- Topics include:
  - Risks of contributing in regulated environments
  - Safe contribution practices
  - Legal and compliance considerations
- Link: https://training.linuxfoundation.org/training/open-source-contribution-in-finance-lfd137/

ii. Beginner's guide to open-source software development (LFD102)

- Topics include:
  - How open-source projects are structured and managed
  - Licensing and collaboration
  - Git, GitHub and CI/CD basics
  - Community norms and best practices
- Link: https://training.linuxfoundation.org/training/beginners-guide-open-source-software-development/

---

9. Final requirements and expectations

By submitting your project, you confirm that:

- It is reasonably easy to install and run using the instructions you provide.
- Your README clearly describes what the project does and how to use it.
- The folder structure and code organisation are clear and navigable.
- The features you claim are implemented and functional.
- You have followed the code quality, security and compliance guidelines above.

Clear documentation, clean code and honest representation of your work are essential requirements for a valid and strong submission.